

Daniel Damazo Kazimoto

RELATÓRIO TÉCNICO

Trabalho FTC - Fundamentos Teóricos da Computação

Relatório técnico apresentado ao
Curso de Ciências da Computação
da matéria FTC, orientado pelo Prof.
Júlio César da Silva

2026

SUMÁRIO

1 INTRODUÇÃO.....	3
2 FUNDAMENTAÇÃO TEÓRICA.....	4
2. 1 Autômato Finito Determinístico.....	4
2. 2 Autômato de Pilha.....	5
2. 3 Máquina de Turing (MT).....	5
3 DESCRIÇÃO DE IMPLEMENTAÇÃO.....	6
3. 1 Autômato Finito Determinístico (AFD).....	6
3. 2 Autômato de Pilha (AP).....	6
3. 3 Máquina de Turing (MT).....	7
5. 1 Testes do Autômato Finito Determinístico (AFD).....	8
5. 2 Testes do Autômato de Pilha (AP).....	8
6 CONCLUSÃO.....	9
REFERÊNCIAS.....	10

1 INTRODUÇÃO

O presente relatório técnico discute os principais conceitos e modelos formais relacionados à Teoria da Computação, focando no Autômato Finito Determinístico (AFD), Autômato de Pilha (AP), Máquina de Turing (MT) e implementação prática de simuladores com a linguagem C#. Além disso, discute os critérios apresentados por Chomsky para a hierarquia de Chomsky, o não-determinismo no reconhecimento de palíndromos e a computação unária de funções aritméticas e seu efeito na complexidade gramatical, limites de computabilidade e na capacidade de armazenamento de memória das máquinas abstratas. São também apresentados logs de execução de cadeias de teste reais, informando como os simuladores desenvolvidos processam lotes de cadeias de teste, otimizando o rastro computacional e contribuindo, conseqüentemente, com maior robustez, fidelidade teórica e confiabilidade de software.

2 FUNDAMENTAÇÃO TEÓRICA

2. 1 Autômato Finito Determinístico

O Autômato Finito Determinístico é o modelo computacional de menor complexidade na Hierarquia de Chomsky, projetado para reconhecer exclusivamente a classe das linguagens regulares (Tipo 3). O determinismo inerente a esse modelo implica que, para cada estado atual e para cada símbolo lido a partir do alfabeto de entrada, existe exatamente uma única transição possível para um próximo estado, impossibilitando bifurcações ou escolhas ambíguas em tempo de processamento.

Função de Transição (δ): As transições são regras que determinam a mudança dos estados internos a partir de um caractere lido na fita. A condição determinística significa que, dado que dois estados e dois símbolos, o estado de destino do próximo símbolo é sempre o mesmo.

Conjunto de Estados (Q): Conjunto de todos os estados de controle internos que o autômato pode alcançar, durante sua execução. Eles atuam como a memória histórica de curto prazo do sistema.

Alfabeto (Σ): Conjunto finito de símbolos físicos permitidos que formam as cadeias de caracteres de entrada (no projeto, $\Sigma = \{ a, b \}$).

Estado Inicial (q_0): Componente que especifica o estado inicial que é obrigatório para a execução do fluxo do autômato.

Estados Finais (F): Subconjunto de estados que determina a validação com sucesso das linguagens regulares. Se a leitura terminar sobre eles, a cadeia é aceita.

2. 2 Autômato de Pilha

Os autômatos de pilha são estruturas que armazenam, processam e fornecem validação de estruturas sintáticas por uma memória auxiliar linear. Ele

mantém um controle intermediário entre os caracteres da entrada e os dados que estão retidos em memória LIFO (Last-In, First-Out), atendendo às operações de empilhamento e desempilhamento, para processar Linguagens Livres de Contexto.

Fita: Sequências de caracteres que o usuário coloca (anbn ou palíndromos)

Pilha: Estrutura de dados vertical que serve para armazenar, hospedar e remover símbolos auxiliares para realizar contagens e verificar simetrias.

Regra de Base (Z0): Regra que determina o fundo da pilha (prevenindo a falha de leitura em pilha vazia) e marca o início ou fim de rastreamento.

Transição Vazia (ϵ -movimento): Permite a mudança de estados ou manipulação da pilha sem consumir caracteres da fita de entrada.

Não-Determinismo: mecanismo recursivo, em L3, que divide o fluxo operacional em vários fluxos, que se executam ao mesmo tempo, para localizar a linha central do eixo dos palíndromos, que não tem delimitador, explicitamente, declarado.

2. 3 Máquina de Turing (MT)

Máquinas de Turing são elementos universais de computação capazes de armazenar, processar, modificar e recuperar dados, usando uma máquina linear expansível. Ela serve como um intermediário universal de computabilidade, trabalhando com linguagens sensíveis ao contexto.

Fita Dinâmica: Infraestrutura principal da máquina, que consiste de um dicionário bidirecional infinitamente expansível, e que interliga células de leitura e escrita sem restrição de tamanho.

Cabeçote: Dispositivo móvel que controla as operações de leitura e gravação de dados, que serve de intermediário físico transferindo dados à Esquerda ou Direita que se movem célula por célula.

Estados Terminais: Estados de parada definitivos, que terminam o processamento e emitem a sentença final do sistema.

3 DESCRIÇÃO DE IMPLEMENTAÇÃO

3. 1 Autômato Finito Determinístico (AFD)

O simulador de AFD foi projetado como um motor de processamento sequencial de strings orientado por uma matriz de estados. Na arquitetura do software desenvolvido em C#, as decisões de projeto basearam-se na criação de dicionários aninhados para mapear de forma estática e imutável a função total de transição delta definida na modelagem matemática.

Determinismo Estrito – A busca na estrutura de dados retorna um único estado de destino de forma direta, impedindo qualquer bifurcação de fluxo ou ambiguidade em tempo de execução.

Consumo de Fita – A cadeia de caracteres de entrada é iterada caractere por caractere por meio de um laço de repetição, atualizando dinamicamente a variável de estado atual até a exaustão dos símbolos.

3. 2 Autômato de Pilha (AP)

O simulador de Autômato de Pilha foi implementado estendendo as capacidades do módulo anterior através da incorporação da classe nativa (Stack). Esta estrutura de dados vertical gerencia os símbolos auxiliares do alfabeto da pilha, operando estritamente sob a disciplina LIFO (Last-In, First-Out) para validar o histórico de dados e checar as restrições sintáticas exigidas.

Gerenciamento de Memória Auxiliar – O motor do software responde instantaneamente aos comandos de empilhamento (Push) e remoção (Pop), simulando as operações físicas na base e no topo da pilha a cada transição de estado efetuada.

Exploração Não-Determinística – Para o reconhecimento de palíndromos, o algoritmo utiliza chamadas recursivas e estruturas de busca em profundidade, permitindo que o simulador explore caminhos paralelos e identifique o eixo central da cadeia sem travar o sistema

3. 3 Máquina de Turing (MT)

A Máquina de Turing foi desenvolvida como o componente mais complexo do sistema, emulando um dispositivo universal de computação. No código-fonte, a infraestrutura da fita magnética bidirecional infinita foi modelada através de uma

lista dinâmica indexável (List<char>), associada a um ponteiro numérico inteiro que controla a fita com precisão, apontando para as coordenadas de leitura e gravação do cabeçote.

Fita Dinâmica Expansível: Simulador expande a lista automaticamente à esquerda ou à direita, e o ponteiro do cabeçote sempre atinge o limite atual da lista de células de memória quando ele atinge a esquerda ou à direita.

Operação Bidirecional: O ponteiro do cabeçote executa operações aritméticas de incremento para deslocamentos à Direita (R) e decremento para deslocamentos à Esquerda (L).

Controle de Parada: As transições para qaccept ou qreject, que representam estados imutáveis, encerram o fluxo operacional ao ser interceptados, imprimindo o veredito final e exibindo o conteúdo na fita.

4 RESULTADOS E TESTES

Execução do lote de testes obrigatórios para a linguagem L1 (cadeias que terminam com "ab").

Cadeia de Entrada	Resultado Esperado	Resultado Obtido
ab	ACEITA	ACEITA
aab	REJEITADA	REJEITADA
bab	ACEITA	ACEITA
ababab	ACEITA	ACEITA
ba	REJEITADA	REJEITADA

E	REJEITADA	REJEITADA
b	REJEITADA	REJEITADA

5 ANÁLISE COMPARATIVA

5.1 Testes do Autômato Finito Determinístico (AFD)

O que pode reconhecer: O AFD é capaz de reconhecer apenas Linguagens Regulares. Ele é perfeito para buscar padrões estáticos, prefixos ou sufixos fixos, como é o caso da linguagem (cadeias terminadas em "ab"). Ele consegue lembrar que "acabou de ver a", transitando para um estado de expectativa.

O que não pode reconhecer: O AFD é matematicamente incapaz de realizar contagens arbitrárias ou pareamentos simétricos. Ele falha em reconhecer a linguagem porque, possuindo um número finito de estados predefinidos, não tem como memorizar quantos caracteres 'a' foram lidos caso o valor de seja maior que a sua quantidade de estados.

5.2 Testes do Autômato de Pilha (AP)

O que pode reconhecer: O AP é projetado para reconhecer Linguagens Livres de Contexto. Ele reconhece perfeitamente a linguagem ($a^n b^n$) empilhando os "a" e desempilhando-os contra os "b". Adicionado ao mecanismo de não-determinismo, ele também consegue reconhecer as simetrias complexas dos palíndromos da linguagem.

O que não pode reconhecer: O AP não consegue lidar com múltiplas dependências cruzadas ou contagens simultâneas. Ele falha categoricamente em reconhecer a linguagem ($a^n b^n c^n$). Ao utilizar a pilha para parear o "a" com o "b", a memória é totalmente esvaziada. Quando os caracteres "c" começam a ser lidos, a máquina não possui mais o registro original, tornando a validação do terceiro bloco impossível.

5.3 Testes da Máquina de Turing (MT)

O que pode reconhecer: A MT reconhece Linguagens Sensíveis ao Contexto e Linguagens Recursivamente Enumeráveis. O acesso bidirecional

permite que o cabeçote varra a fita múltiplas vezes, resolvendo facilmente a linguagem ($a^n b^n c^n$) através da técnica de marcação de símbolos. Além do reconhecimento sintático, ela realiza cômputo ativo de algoritmos, como processar o incremento da função aritmética.

O que não pode reconhecer: Embora seja o modelo de computabilidade absoluto (Tese de Church-Turing), a MT não pode reconhecer ou decidir linguagens que representam problemas matematicamente indecidíveis. O exemplo clássico é o Problema da Parada (Halting Problem), nenhuma Máquina de Turing pode receber o código de outra MT e prever, com 100% de precisão, se ela vai parar de funcionar ou se entrará em um loop infinito.

6 CONCLUSÃO

A implementação progressiva do Autômato Finito Determinístico, do Autômato de Pilha e da Máquina de Turing, na prática, mostrou os limites estabelecidos pela Hierarquia de Chomsky. Ao analisar os problemas que se pode resolver com um algoritmo muda radicalmente, de validação de sufixo (AFD) para processamento recursivo de palíndromos (AP), ou para a marcação dinâmica de linguagens sensíveis ao contexto (MT).

Como reflexão fundamental, um dos maiores questionamentos da ciência da computação: **Um computador moderno é mais poderoso que uma Máquina de Turing?** À luz da Tese de Church-Turing, a resposta teórica é não. A tese postula que qualquer algoritmo executável ou função computável no mundo real pode ser simulado por uma Máquina de Turing universal. Os computadores modernos baseados na arquitetura de Von Neumann superam a Máquina de Turing de forma esmagadora em velocidade, eficiência energética, paralelismo e capacidade de processamento empírico. No entanto, sob a ótica lógica da computabilidade, eles possuem exatamente o mesmo poder.

REFERÊNCIAS

1. HOPCROFT, John E.; MOTWANI, Rajeev; ULLMAN, Jeffrey D. Introdução à Teoria de Autômatos, Linguagens e Computação. 2. ed. Rio de Janeiro: Campus/Elsevier, 2003
2. MENEZES, Paulo Blauth. Linguagens Formais e Autômatos. 6. ed. Porto Alegre: Bookman, 2010
3. VIEIRA, Newton José. Introdução aos Fundamentos da Computação: Linguagens e Máquinas. São Paulo: Pioneira Thomson Learning, 2006